Feed-forward neural networks: from the neuron to the multilayer perceptron

Prof. Dr. Emre Onur Kahya · Department of Physics Engineering, Istanbul Technical University

Logistic regression is a single artificial neuron, and it draws a hyperplane. Note 10 obtained curved boundaries by fixing a non-linear feature map in advance; a neural network instead *learns* the feature map, by stacking layers of neurons and fitting all their weights to the data. This note introduces the artificial neuron and the activation functions in use today (sigmoid, tanh, ReLU, GELU, SiLU), shows how one hidden layer solves XOR, writes the forward pass of a multilayer perceptron in matrix form, and matches output units to losses for each task. It then asks what such networks can represent — the universal approximation theorem, and why depth can be exponentially more economical than width — and which hyperparameters control their capacity. How the gradients are computed (backpropagation) is the subject of Note 12.

1 From biological to artificial neurons

A nerve cell receives signals from other cells at thousands of contact points, the *synapses*, on its branched *dendrites*. Each incoming signal shifts the electrical potential of the cell body by an amount that depends on the strength of the synapse and may be excitatory or inhibitory. When the summed potential exceeds a threshold, the cell fires: a voltage pulse, the action potential, travels down its *axon* to the synapses of the next cells. The human brain holds some 10^{11} neurons with of the order of 10^3 to 10^4 synapses each, and learning is believed to work largely by changing synaptic strengths.

The artificial neuron keeps only the arithmetic of this picture: a weighted sum of inputs and a threshold-like response. Given an input $\mathbf{x} \in \mathbb{R}^d$, weights $\mathbf{w} \in \mathbb{R}^d$ and a bias $b \in \mathbb{R}$, it computes

$$a = \mathbf{w}^{\mathsf{T}}\mathbf{x} + b = \sum_{j=1}^{d} w_j x_j + b, \qquad y = g(a). \tag{1}$$

The number a is the *pre-activation*, and the fixed function $g : \mathbb{R} \to \mathbb{R}$ is the *activation function* (also called the transfer function or non-linearity). Each weight w_j says how strongly, and with which sign, input j pushes the output; the bias plays the role of a (negative) threshold. Figure 1 shows the usual drawing. The biological analogy should not be pushed further: real neurons communicate by spike trains in continuous time, and the networks of this note are mathematical functions, not brain models.

Three neurons we already know. The choice of g turns (1) into familiar models:

activation $g(a)$	output	model
identity, $g(a) = a$	a real number	linear regression (Note 8)
step, $g(a) = \operatorname{sign}(a)$	± 1	perceptron (Note 9, Section 3)
sigmoid, $g(a) = \sigma(a) = 1/(1 + e^{-a})$	a probability	logistic regression (Note 9, Section 4)

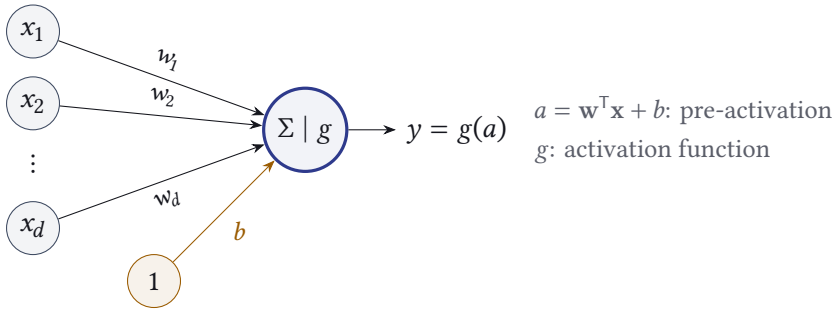


Figure 1. An artificial neuron. The bias can be drawn as the weight of a constant input equal to 1.

The neuron is therefore nothing new. What is new in this note is what happens when many of them are combined.

A neuron is a ridge function. The output depends on $\mathbf{x}$ only through the single number $\mathbf{w}^\mathsf{T}\mathbf{x}$. Moving $\mathbf{x}$ in any direction $\mathbf{u}$ orthogonal to $\mathbf{w}$ leaves a, and hence y, unchanged: the level sets of a neuron are the hyperplanes $\mathbf{w}^\mathsf{T}\mathbf{x} + b = \text{const}$, and along $\mathbf{w}$ the output follows the profile of g. Seen as a landscape over the input space, a neuron is a ridge or a cliff (Figure 6 of Section 6.1, left).

Worked example — 1. One neuron, five activations

Take $\mathbf{x} = (2, 1, 0.5)$, $\mathbf{w} = (0.5, -1, 2)$ and $b = -0.5$. The pre-activation is

$$a = 0.5 \cdot 2 + (-1) \cdot 1 + 2 \cdot 0.5 - 0.5 = 1 - 1 + 1 - 0.5 = 0.5.$$

The identity neuron outputs 0.5; the step neuron outputs +1; the sigmoid neuron outputs $\sigma(0.5) = 1/(1+e^{-0.5}) = 0.622$; a tanh neuron outputs $\tanh 0.5 = 0.462$; a ReLU neuron (Section 2) outputs $\max(0, 0.5) = 0.5$. All five give a positive answer, because $a > 0$; they differ in how the answer depends on a. Now move the input along $\mathbf{u} = (2, 1, 0)$, for which $\mathbf{w}^\mathsf{T}\mathbf{u} = 1 - 1 + 0 = 0$: the point $\mathbf{x} + s\,\mathbf{u} = (2 + 2s, 1 + s, 0.5)$ has $a = 0.5$ for every s, so every neuron gives the same output all along this line — the ridge property.

2 Activation functions

Why a non-linearity at all. Suppose every neuron were linear, $g(a) = a$, and we fed the outputs of a first layer of neurons, $\mathbf{h} = W_1\mathbf{x} + \mathbf{b}_1$, into a second, $\mathbf{y} = W_2\mathbf{h} + \mathbf{b}_2$. Then

$$\mathbf{y} = W_2(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (W_2W_1)\,\mathbf{x} + (W_2\mathbf{b}_1 + \mathbf{b}_2),$$

again a single linear map: stacking linear layers adds parameters but no expressive power, and any number of layers collapses to one. Everything interesting in a neural network comes from the non-linear g between the layers.

The classical choices. Table 1 lists the activation functions met in practice and Figure 2 plots them with their derivatives. The *logistic sigmoid* σ and the *hyperbolic tangent* were the standard choices until about 2010. They are smooth, monotone and bounded, and they *saturate*: for $|a|$ larger than a few units the output is nearly constant and the derivative nearly zero. The two are the same function up to scaling,

$$\tanh a = \frac{e^a - e^{-a}}{e^a + e^{-a}} = \frac{1 - e^{-2a}}{1 + e^{-2a}} = \frac{2}{1 + e^{-2a}} - 1 = 2\sigma(2a) - 1, \tag{2}$$

so a network of tanh units can be rewritten as a network of sigmoid units with rescaled weights (Question C1). The differences are practical: tanh is zero-centred, $\tanh 0 = 0$, and its slope at the origin is 1, four times the sigmoid's maximal slope $\sigma'(0) = \tfrac{1}{4}$. Both derivatives follow from the function value alone,

$$\sigma'(a) = \sigma(a)\big(1-\sigma(a)\big), \qquad \tanh'(a) = \frac{(e^{a}+e^{-a})^{2}-(e^{a}-e^{-a})^{2}}{(e^{a}+e^{-a})^{2}} = 1-\tanh^{2} a,$$

the first derived in Note 9, Section 4.1. That a derivative can be computed from the stored output is convenient for backpropagation (Note 12, Section 3.2).

Table 1. Activation functions. Φ and φ are the standard normal distribution function and density.

name	$g(a)$	$g'(a)$	range	introduced / popularised
logistic sigmoid	$\sigma(a) = 1/(1+e^{-a})$	$\sigma(a)(1-\sigma(a))$	$(0,1)$	classical
hyperbolic tangent	$\tanh a$	$1-\tanh^{2} a$	$(-1,1)$	classical
ReLU	$\max(0,a)$	$\mathbf{1}\{a>0\}$	$[0,\infty)$	[26, 8]
leaky ReLU	$\max(\alpha a, a)$, $0<\alpha<1$	1 or α	$\mathbb{R}$	[21]
GELU	$a\,\Phi(a)$	$\Phi(a)+a\,\varphi(a)$	$[-0.17,\infty)$	[12]
SiLU (swish)	$a\,\sigma(a)$	$\sigma(a)\big(1+a(1-\sigma(a))\big)$	$[-0.28,\infty)$	[7, 30]

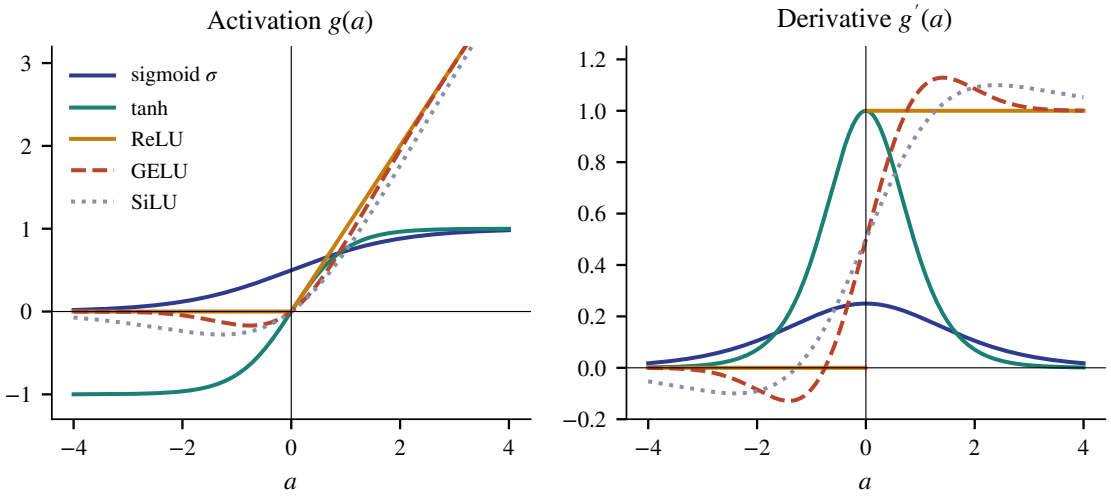


Figure 2. The activation functions of Table 1 (left) and their derivatives (right). Sigmoid and tanh saturate on both sides; ReLU has slope exactly 1 for $a>0$ and 0 for $a<0$; GELU and SiLU are smooth versions of ReLU that dip slightly below zero for negative a.

The rectified linear unit. The ReLU, $\mathrm{ReLU}(a) = \max(0,a)$ [26, 8], became the default for deep networks in the early 2010s. Its derivative is exactly 1 for every positive input, so it does not saturate on that side, and it costs a single comparison. For any given input a sizeable fraction of ReLU units output exactly zero, so the representation is *sparse*. The price is the other side: a unit whose pre-activation is negative for every training input receives zero gradient and its own weights stop changing; unless the layers below happen to shift its inputs, it stays silent — a *dead* unit. The *leaky ReLU* keeps a small slope α (say 0.01) for $a<0$ to avoid this; in the *parametric* ReLU (PReLU) [10] α is itself learned with the weights. The kink at $a=0$ is harmless: the derivative is undefined at a single point, which training almost never hits exactly, and libraries simply use $g'(0)=0$.

Smooth rectifiers. The *Gaussian error linear unit* [12], $\mathrm{GELU}(a) = a\,\Phi(a)$, multiplies the input by the probability that a standard normal variable lies below it: large positive inputs pass almost unchanged ($\Phi \approx 1$), large negative ones are suppressed ($\Phi \approx 0$), and in between the gate is soft. The *sigmoid-weighted linear unit* $\mathrm{SiLU}(a) = a\,\sigma(a)$ [7], also called swish [30], does the same with the logistic gate. Both are smooth and slightly non-monotone, with minima -0.170 at $a \approx -0.75$ (GELU) and -0.278 at $a \approx -1.28$ (SiLU). Since $\Phi(a) \approx \sigma(1.702\,a)$, the two curves are close; the tanh-based approximation $\frac{1}{2}a\big(1+\tanh[\sqrt{2/\pi}\,(a+0.044715\,a^3)]\big)$ of GELU, used in several implementations, gives 0.84119 at $a = 1$ against the exact 0.84134.

Worked example — 2. Reading the activations

At a few pre-activations the functions and their derivatives take the values

a	$\sigma(a)$	$\sigma'(a)$	$\tanh a$	$\tanh' a$	$\mathrm{ReLU}(a)$	GELU	GELU′	SiLU	SiLU′
−2	0.119	0.105	−0.964	0.071	0	−0.046	−0.085	−0.238	−0.091
−1	0.269	0.197	−0.762	0.420	0	−0.159	−0.083	−0.269	0.072
0	0.500	0.250	0	1.000	0	0	0.500	0	0.500
1	0.731	0.197	0.762	0.420	1	0.841	1.083	0.731	0.928
2	0.881	0.105	0.964	0.071	2	1.954	1.085	1.762	1.091

Three checks. (i) Equation (2) at $a = 1$: $2\sigma(2) - 1 = 2(0.881) - 1 = 0.762 = \tanh 1$. (ii) $\sigma'(1) = 0.731 \times 0.269 = 0.197$ and $\tanh'(1) = 1 - 0.762^2 = 0.420$: the derivatives follow from the outputs. (iii) At $a = 2$ the sigmoid passes on only 0.105 of a small change in its input and tanh only 0.071, while ReLU, GELU and SiLU pass on about 1: in a chain of layers these factors multiply (Note 12, Section 9.2), which is why the saturating functions make deep networks hard to train. Note also that $\mathrm{SiLU}(a) - \mathrm{SiLU}(-a) = a\sigma(a) + a\sigma(-a) = a$, the same identity as $\mathrm{ReLU}(a) - \mathrm{ReLU}(-a) = a$: both are "half of the identity" plus an even correction.

Softmax, an output activation. The functions above act on each pre-activation separately. The *softmax* acts on a whole vector $\mathbf{a} \in \mathbb{R}^m$ and returns a probability vector,

$$\mathrm{softmax}(\mathbf{a})_k = \frac{e^{a_k}}{\sum_{j=1}^{m} e^{a_j}}, \qquad k = 1, \dots, m, \tag{3}$$

with positive entries summing to one. It is the output layer of every multiclass network. Note 9, Section 4.4 derived it as multiclass logistic regression, with cross-entropy gradient $\mathbf{y} - \mathbf{t}$; because adding the same constant to all scores changes nothing, implementations subtract $\max_j a_j$ before exponentiating. Dividing the scores by a *temperature* $T > 0$, $y_k \propto e^{a_k/T}$, sharpens ($T < 1$) or flattens ($T > 1$) the distribution (Note 2, Section 2.2): the scores $\mathbf{a} = (2, 1, 0)$ give $(0.665, 0.245, 0.090)$ at $T = 1$, $(0.867, 0.117, 0.016)$ at $T = 0.5$ and $(0.367, 0.332, 0.301)$ at $T = 10$. The flattened outputs carry information that a one-hot label does not: they say which wrong classes the network finds plausible. *Knowledge distillation* exploits this by training a small "student" network to match the softened ($T > 1$) outputs of a large "teacher" [14].

In physics — activation functions are thermal averages

The functions of this section are old friends from statistical mechanics. A spin $s = \pm 1$ in a local field B (measured in energy units), with energy $-Bs$ at temperature T, has Boltzmann probabilities proportional to $e^{\pm\beta B}$ with $\beta = 1/k_B T$, and its mean is

$$\langle s \rangle = \frac{e^{\beta B} - e^{-\beta B}}{e^{\beta B} + e^{-\beta B}} = \tanh(\beta B) \ :$$

a tanh unit reports the mean orientation of a spin in the field a, with β absorbed into the weights. The mean-field equation of the Ising model, $\bar{s} = \tanh\big(\beta(Jq\bar{s} + B)\big)$ for the mean spin $\bar{s}$ and q neighbours, is a single tanh unit whose output is fed back as its own input. The occupation of a fermion level of energy E, $1/(e^{(E-\mu)/k_BT} + 1) = \sigma\big(-(E-\mu)/k_BT\big)$, is the logistic sigmoid of the Fermi–Dirac distribution, which becomes a step function as $T \to 0$. The softmax is the Boltzmann distribution over m states with energies $E_k = -a_k$ at $k_BT = 1$; its denominator is the partition function, so $\log \sum_k e^{a_k}$ is minus a free energy, and the softmax temperature T is literally a temperature. Hopfield's 1982 network [15], a landmark of the field, made the link explicit: it is an Ising model whose couplings store memories.

Modern connection — the activations of large language models

BERT and GPT-2 use GELU in their feed-forward blocks [6, 28]. Many recent language models use a *gated* variant, SwiGLU [33]: instead of $W_2\, g(W_1\mathbf{x})$ the block computes $W_2\big(\mathrm{SiLU}(W_1\mathbf{x}) \odot (V\mathbf{x})\big)$, where $\odot$ is the element-wise product and the second matrix V produces a learned, input-dependent gate. LLaMA is an example [35]; because the gated block has three weight matrices instead of two, LLaMA shrinks its hidden width from $4d$ to $\frac{2}{3} \cdot 4d$ to keep the number of parameters unchanged. These choices were made empirically, by comparing validation losses of otherwise identical models; the theory of this note does not single out a best activation, only the requirement that it be non-linear (and not a polynomial, Section 6.2).

3 Learning the features

A sigmoid neuron still draws a hyperplane (Note 9, Section 4.1), as does any monotone g thresholded at a level it crosses once, and no hyperplane solves XOR (Note 9, Section 6). Of the "three routes to non-linearity" of Note 10, Section 2, the first two compute a map chosen in advance or use it only through its kernel; neural networks take the third and fit the map itself to the data. Their first layer computes features

$$\mathbf{h} = g(W_1\mathbf{x} + \mathbf{b}_1) \in \mathbb{R}^{H_1}, \qquad \text{i.e.} \quad h_j = g\big(\mathbf{w}_{1,j}^{\mathsf{T}}\mathbf{x} + b_{1,j}\big), \quad j = 1, \dots, H_1, \tag{4}$$

with g applied element-wise, and the second layer is a linear model on $\mathbf{h}$ — for classification, logistic or softmax regression. The difference from Note 10 is that W_1 and $\mathbf{b}_1$ are not fixed in advance: they are fitted to the data together with the output weights. Each hidden unit h_j is a neuron, so a learned feature is a ridge function of the input, and the network chooses the directions $\mathbf{w}_{1,j}$ in which the ridges run.

Worked example — 3. Two hidden units solve XOR

Use the 0/1 form of XOR: $t = 1$ for $(0,1)$ and $(1,0)$, $t = 0$ for $(0,0)$ and $(1,1)$. Take two ReLU hidden units that look at the same direction with different thresholds,

$$h_1 = \mathrm{ReLU}(x_1 + x_2), \qquad h_2 = \mathrm{ReLU}(x_1 + x_2 - 1), \qquad y = h_1 - 2h_2,$$

that is $W_1 = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$, $\mathbf{b}_1 = (0, -1)$, output weights $(1, -2)$ and output bias 0.

$\mathbf{x}$	t	$x_1 + x_2$	(h_1, h_2)	$y = h_1 - 2h_2$	$y > \frac{1}{2}$?
$(0,0)$	0	0	$(0,0)$	0	no
$(0,1)$	1	1	$(1,0)$	1	yes
$(1,0)$	1	1	$(1,0)$	1	yes
$(1,1)$	0	2	$(2,1)$	0	no

The network reproduces XOR exactly. Figure 3 shows why. The hidden layer maps the four inputs to only three points: $(0,1)$ and $(1,0)$ land on the same point $(1,0)$, and $(1,1)$ is lifted to $(2,1)$. In the hidden space (h_1, h_2) the classes are linearly separable, by the line $h_1 - 2h_2 = \frac{1}{2}$, and the output unit is an ordinary linear classifier there. Read back in the input space, the output is the piecewise-linear function of $s = x_1 + x_2$ that rises from 0 to 1 on $0 \le s \le 1$ and falls back to 0 on $1 \le s \le 2$; thresholding it at $\frac{1}{2}$ keeps the band $\frac{1}{2} < x_1 + x_2 < \frac{3}{2}$, which contains exactly the two class-1 points. These weights were set by hand. Gradient training from random initial values often finds an equivalent solution, although with only two hidden units it can also get stuck (Section 7.3; Note 12, Section 9).

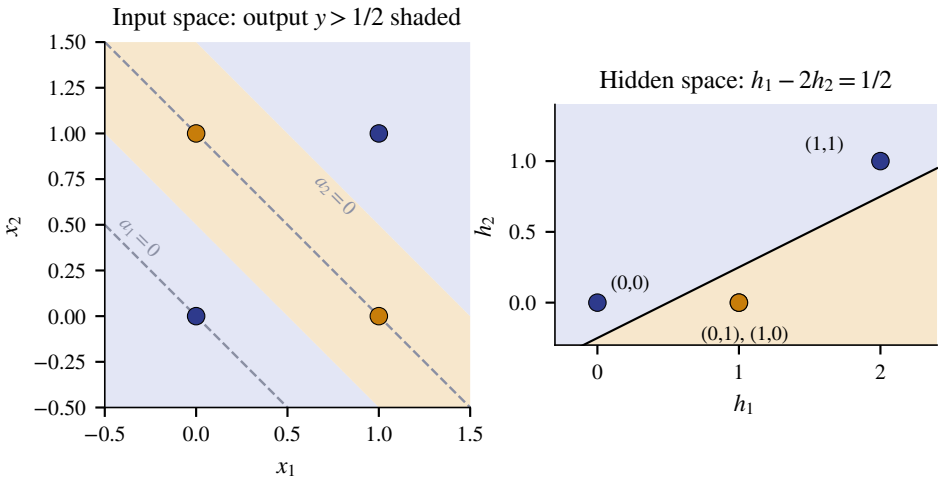


Figure 3. The XOR network of Worked example 3. Left: input space; the dashed lines are where the hidden pre-activations vanish, and the shaded band is where the output exceeds $\frac{1}{2}$. Right: the hidden representation (h_1, h_2), in which the two inputs of class 1 (amber) coincide and a straight line separates the classes.

Logic gates. With step activations, $g(a) = \mathbf{1}\{a > 0\}$, a neuron is a logic gate: $\mathbf{1}\{x_1 + x_2 - 0.5 > 0\}$ is OR and $\mathbf{1}\{x_1 + x_2 - 1.5 > 0\}$ is AND on binary inputs, and a negative weight implements NOT. XOR is "OR and not AND", so two hidden threshold units and one output unit compute it, which is McCulloch and Pitts's 1943 view of neurons as logic elements [22]. Any Boolean function of d inputs is a one-hidden-layer network of threshold units: give each input pattern of class 1 its own hidden unit that fires on that pattern only, and let the output compute their OR. The construction may need up to 2^d hidden units, an early warning that "possible" and "economical" differ (Section 6.3).

4 The multilayer perceptron

4.1 One hidden layer

A *multilayer perceptron* (MLP) with one hidden layer of width H_1 and m outputs computes

$$\mathbf{h} = g\big(W_1\mathbf{x} + \mathbf{b}_1\big), \qquad \mathbf{y} = o\big(W_2\mathbf{h} + \mathbf{b}_2\big), \qquad W_1 \in \mathbb{R}^{H_1 \times d},\ \mathbf{b}_1 \in \mathbb{R}^{H_1},\ W_2 \in \mathbb{R}^{m \times H_1},\ \mathbf{b}_2 \in \mathbb{R}^{m}, \tag{5}$$

where g is the hidden activation, applied element-wise, and o the output activation (Section 5). Written out for one output, $f(\mathbf{x}) = o\big(\sum_{j=1}^{H_1} w_{2,j}\, g(\mathbf{w}_{1,j}^{\mathsf{T}}\mathbf{x} + b_{1,j}) + b_2\big)$: a weighted sum of H_1 ridge functions. The parameters are collected in $\theta = \{W_1, \mathbf{b}_1, W_2, \mathbf{b}_2\}$, and their number is

$$p = H_1(d+1) + m(H_1+1). \tag{6}$$

The inputs form the *input layer*, the h_j the *hidden layer* (hidden because the data never specify their values), and $\mathbf{y}$ the *output layer* (Figure 4). Every unit of a layer receives every unit of the layer below, so these layers are called *fully connected* or *dense*. The network is *feed-forward*: the connections form no cycle, and information flows from input to output in one pass. The name "perceptron" is historical; the units are not perceptrons with step functions but smooth or piecewise-linear neurons, which is what makes gradient training possible. Counting conventions differ: this network has one hidden layer but two layers of weights, and is called a "two-layer network" in some books.

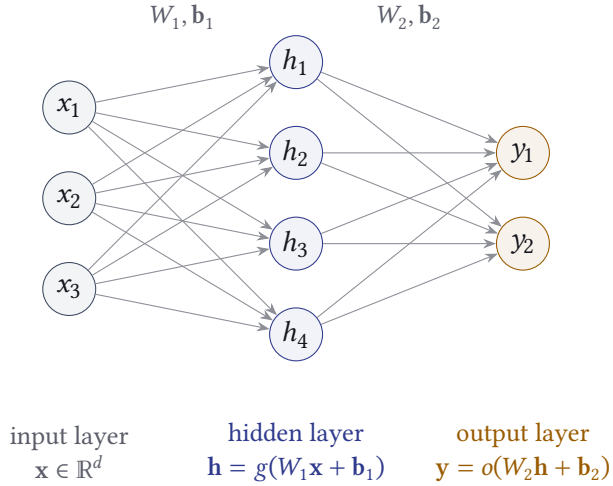


Figure 4. A multilayer perceptron with $d = 3$ inputs, one hidden layer of $H_1 = 4$ units and $m = 2$ outputs. Each arrow carries one weight; biases are not drawn. By (6) it has $4 \cdot 4 + 2 \cdot 5 = 26$ parameters.

Sums of ridge functions can be surprisingly flexible: a handful of hidden units already builds a localised bump, and sums of bumps approximate any reasonable function (Section 6).

Matrix form for a mini-batch. Networks are evaluated on many examples at once. Stack the n' inputs of a mini-batch as the rows of $X \in \mathbb{R}^{n' \times d}$, as in Note 1. The forward pass of (5) becomes

$$A_1 = XW_1^{\mathsf{T}} + \mathbf{1}\mathbf{b}_1^{\mathsf{T}}, \quad H = g(A_1), \quad A_2 = HW_2^{\mathsf{T}} + \mathbf{1}\mathbf{b}_2^{\mathsf{T}}, \quad Y = o(A_2), \tag{7}$$

with $H \in \mathbb{R}^{n' \times H_1}$ and $Y \in \mathbb{R}^{n' \times m}$; row i of each matrix belongs to example i, and the products with $\mathbf{1} \in \mathbb{R}^{n'}$ add the bias to every row ("broadcasting" in numpy and PyTorch). Each layer is one matrix product followed by an element-wise function, exactly the operations that GPUs execute fastest.

Worked example — 4. A forward pass on a mini-batch of two

A network with $d = 2$ inputs, $H_1 = 2$ ReLU hidden units and a softmax over $m = 3$ classes has

$$W_1 = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}, \quad \mathbf{b}_1 = \begin{pmatrix} 0 \\ -1 \end{pmatrix}, \qquad W_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ -1 & 1 \end{pmatrix}, \quad \mathbf{b}_2 = \mathbf{0}.$$

The mini-batch holds $\mathbf{x}_1 = (2, 1)$ of class 2 and $\mathbf{x}_2 = (1, 0)$ of class 1.

Hidden layer. Row by row, XW_1^{T} contains the dot products of each input with the rows of W_1:

$$X = \begin{pmatrix} 2 & 1 \\ 1 & 0 \end{pmatrix}, \quad A_1 = XW_1^{\mathsf{T}} + \mathbf{1}\mathbf{b}_1^{\mathsf{T}} = \begin{pmatrix} 2-1 & 2+1 \\ 1-0 & 1+0 \end{pmatrix} + \begin{pmatrix} 0 & -1 \\ 0 & -1 \end{pmatrix} = \begin{pmatrix} 1 & 2 \\ 1 & 0 \end{pmatrix} = H.$$

All pre-activations are non-negative, so the ReLU leaves them unchanged; the second hidden unit sits exactly at its threshold for $\mathbf{x}_2$ and outputs zero.
Output layer. $A_2 = HW_2^{\mathsf{T}}$: for $\mathbf{x}_1$, $(1, 2, -1+2) = (1, 2, 1)$; for $\mathbf{x}_2$, $(1, 0, -1+0) = (1, 0, -1)$. The softmax gives

$$\mathbf{y}_1 = \frac{(e, e^2, e)}{2e + e^2} = (0.212,\ 0.576,\ 0.212), \qquad \mathbf{y}_2 = (0.665,\ 0.245,\ 0.090).$$

The second row equals the softmax of $(2, 1, 0)$ computed in Note 9, Section 4.4, because the scores differ by the constant 1.
Loss. The cross-entropies are $-\log 0.576 = 0.551$ and $-\log 0.665 = 0.408$, and the mini-batch loss is their mean, 0.480. Both examples are classified correctly (the largest probability belongs to the true class), but neither confidently: training would push these probabilities towards 1.

4.2 Deep networks

Nothing stops us from feeding the hidden layer into another hidden layer. A *deep* MLP with L layers of weights and widths $H_0 = d, H_1, \dots, H_{L-1}, H_L = m$ computes, starting from $\mathbf{h}^{(0)} = \mathbf{x}$,

$$\mathbf{a}^{(k)} = W^{(k)}\mathbf{h}^{(k-1)} + \mathbf{b}^{(k)}, \qquad \mathbf{h}^{(k)} = g\big(\mathbf{a}^{(k)}\big), \qquad k = 1, \dots, L-1, \qquad \mathbf{y} = o\big(W^{(L)}\mathbf{h}^{(L-1)} + \mathbf{b}^{(L)}\big), \tag{8}$$

with $W^{(k)} \in \mathbb{R}^{H_k \times H_{k-1}}$ and $\mathbf{b}^{(k)} \in \mathbb{R}^{H_k}$. The function is a composition, $f = f^{(L)} \circ \dots \circ f^{(1)}$, of simple layer maps, and the number of parameters is

$$p = \sum_{k=1}^{L} H_k\,(H_{k-1} + 1). \tag{9}$$

The same letters are used in Notes 12 and 13: layer index k, widths H_k, pre-activations $\mathbf{a}^{(k)}$, activations $\mathbf{h}^{(k)}$, activation function g, and ℓ for the loss of one example. The intermediate vectors $\mathbf{h}^{(k)}$ are the network's internal *representations* of the input. In trained image networks, the early layers respond to local edges and colour contrasts, intermediate layers to textures and object parts, and late layers to whole objects (Note 13, Section 7, shows how such features are visualised); each layer re-describes the input in terms of the features found by the layer below. This *hierarchy of representations* is the main idea of deep learning, and it echoes the way physics describes matter: detector hits are combined into tracks, tracks into particles, particles into the event. The forward pass is Algorithm 1; it keeps every intermediate $A^{(k)}$ and $H^{(k)}$, because the backward pass (Note 12, Section 3.4) reuses them.

Running time — forward pass of an MLP

Time: layer k multiplies an $n' \times H_{k-1}$ matrix by a $H_{k-1} \times H_k$ matrix, $n' H_{k-1} H_k$ multiply–adds; the biases and activations add $O(n' H_k)$. In total $O\big(n' \sum_k H_{k-1} H_k\big) = O(n'\,p)$: a few operations per parameter and per example, the assumption made in Note 7, Section 1. *Prediction*: $O(p)$ per example, independent of the number n of training examples — unlike kernel methods (Note 10, Section 9), whose prediction cost grows with n. *Memory*: $O(p)$ for the parameters; during training also $O\big(n' \sum_k H_k\big)$ for the cache of activations, which for wide or deep networks and large mini-batches often exceeds the memory of the parameters themselves.

Algorithm 1 Forward pass and prediction of an L-layer MLP on a mini-batch

Require: mini-batch $X \in \mathbb{R}^{n' \times d}$; weights $W^{(k)} \in \mathbb{R}^{H_k \times H_{k-1}}$ and biases $\mathbf{b}^{(k)}$, $k = 1, \dots, L$; hidden activation g; output activation o

function FORWARD(X; θ)
  $H^{(0)} \leftarrow X$
  for $k = 1, \dots, L$ **do**
    $A^{(k)} \leftarrow H^{(k-1)} W^{(k)\mathsf{T}} + \mathbf{1}\mathbf{b}^{(k)\mathsf{T}}$ ▷ matrix product, $O(n' H_{k-1} H_k)$
    if $k < L$ **then**
      $H^{(k)} \leftarrow g(A^{(k)})$ ▷ element-wise, $O(n' H_k)$
    end if
  end for
  $Y \leftarrow o(A^{(L)})$ ▷ e.g. softmax of each row, $O(n' m)$
  return Y and the cache $\{A^{(k)}, H^{(k)}\}$ ▷ the cache is needed for backpropagation
end function
function PREDICT($\mathbf{x}$; θ)
  $(Y, \cdot) \leftarrow$ FORWARD($\mathbf{x}^{\mathsf{T}}$; θ) ▷ a mini-batch of one; the cache is discarded
  return $\arg\max_j Y_{1j}$ (classification) or Y itself (regression)
end function

Worked example — 5. Counting parameters and operations

A classifier for 28×28 grey-scale digit images has $d = 784$ inputs, hidden layers of widths 512 and 256, and $m = 10$ outputs. By (9),

$$p = 512\,(784 + 1) + 256\,(512 + 1) + 10\,(256 + 1) = 401\,920 + 131\,328 + 2\,570 = 535\,818.$$

All but $512 + 256 + 10 = 778$ of these are weights, so the forward pass of one image costs 535 040 multiply–adds, about a million floating-point operations; a mini-batch of 256 images costs about 2.7×10^8, a fraction of a millisecond on a modern GPU. In 32-bit floats the parameters take $535\,818 \times 4$ bytes ≈ 2.1 MB. The first layer holds 75% of the parameters: dense layers on raw high-dimensional inputs are expensive, one motivation for the convolutional layers of Note 13, which share weights across positions.

Modern connection — the MLP inside every transformer

A transformer layer has two parts: an attention block, which mixes information between the positions of a sequence (Note 5 related attention to kernel smoothing), and an MLP block, which processes each position separately with the same one-hidden-layer network, $\mathbf{h} \mapsto W_2\, g(W_1 \mathbf{h} + \mathbf{b}_1) + \mathbf{b}_2$, of hidden width $4 d_{\text{model}}$ (or a gated version, as in the modern box of Section 2). In GPT-2 small [28], with $d_{\text{model}} = 768$ and GELU, each MLP block has $768 \cdot 3072 + 3072 + 3072 \cdot 768 + 768 = 4\,722\,432$ parameters, against $4\,(768^2 + 768) = 2\,362\,368$ for the four projection matrices of attention: two-thirds of the weights of each of its 12 layers sit in plain MLPs. Each block is wrapped as $\mathbf{h} \leftarrow \mathbf{h} + \text{MLP}(\text{Norm}(\mathbf{h}))$, with a *residual connection* and a *normalisation layer*; these two devices, which make very deep stacks trainable, are introduced in Note 12, Sections 10.2 and 10.3.

5 Output units and losses

The hidden layers can use any activation; the output layer cannot, because its values must have the meaning that the task requires: an unbounded number, a probability, a probability vector. The principled way to choose both the output unit and the loss is maximum likelihood (Note 2, Section 6): let the network output the parameters of a conditional distribution $p(t \mid \mathbf{x})$, and minimise the negative log-likelihood $\ell = -\log p(t \mid \mathbf{x})$ of the observed target. Table 2 lists the standard pairs; the losses themselves were introduced in Note 4, Section 2.

Table 2. Output units and losses. $\mathbf{a} = \mathbf{a}^{(L)}$ is the output pre-activation, $\mathbf{y} = o(\mathbf{a})$ the output. For every row the gradient of the loss with respect to $\mathbf{a}$ is $\mathbf{y} - \mathbf{t}$ (with $\mathbf{t}$ one-hot for the softmax row).

task	target	output unit o	model $p(t \mid \mathbf{x})$	loss ℓ
regression	$\mathbf{t} \in \mathbb{R}^m$	identity	Gaussian, mean $\mathbf{y}$, fixed variance	$\frac{1}{2}\lVert\mathbf{y} - \mathbf{t}\rVert^2$
binary classification	$t \in \{0, 1\}$	sigmoid	Bernoulli(y)	$-t \log y - (1-t)\log(1-y)$
m classes	one-hot $\mathbf{t}$	softmax	categorical($\mathbf{y}$)	$-\sum_k t_k \log y_k$
multi-label	$\mathbf{t} \in \{0, 1\}^m$	m sigmoids	m independent Bernoullis	sum of m binary cross-entropies
counts	$t \in \mathbb{N}$	exponential	Poisson, mean y	$y - t \log y$

Prediction minus target. For each row of Table 2 the derivative of the loss with respect to the output pre-activation is simply $\mathbf{y} - \mathbf{t}$. For the identity and squared loss this is immediate, $\partial\ell/\partial a_k = y_k - t_k$; for the sigmoid with binary cross-entropy and for the softmax it was derived in Note 9, Sections 4.2 and 4.4. These pairs are not a coincidence. Each target distribution of the table can be written in the exponential-family form

$$p(t \mid a) = \kappa(t)\, e^{at - A(a)}, \qquad A(a) = \log \sum_t \kappa(t)\, e^{at}$$

(an integral for continuous t), with a the output pre-activation: Bernoulli, $\kappa = 1$ and $A(a) = \log(1+e^a)$; Poisson, $\kappa(t) = 1/t!$ and $A(a) = e^a$; Gaussian with unit variance, $\kappa(t) = e^{-t^2/2}/\sqrt{2\pi}$ and $A(a) = a^2/2$. The negative log-likelihood is $\ell = A(a) - at - \log \kappa(t)$, and

$$\frac{\partial \ell}{\partial a} = A'(a) - t = \frac{\sum_t t\, \kappa(t)\, e^{at}}{\sum_t \kappa(t)\, e^{at}} - t = \mathbb{E}[t \mid a] - t = y - t,$$

because the output unit of each row returns exactly the mean: $A'(a) = \sigma(a)$, e^a and a in the three cases (for the Poisson row, $\ell = e^a - ta + \log t!$ gives $e^a - t$ directly). A is a log partition function, and its derivative is a mean, exactly as in statistical mechanics (physics box of Section 2); the softmax row is the same statement with a vector $\mathbf{a}$ and $A(\mathbf{a}) = \log \sum_k e^{a_k}$. Statisticians call such an output unit the *canonical link* of a generalised linear model. Note 12, Section 2, uses $\mathbf{y} - \mathbf{t}$ as the starting point of backpropagation.

Squared loss on a sigmoid output. Should a sigmoid output be trained with the squared loss $\ell = \frac{1}{2}(y - t)^2$ or with the cross-entropy? With the squared loss the chain rule gives

$$\frac{\partial \ell}{\partial a} = (y - t)\, \sigma'(a) = (y - t)\, y\, (1 - y), \tag{10}$$

which carries the factor $y(1-y) \le \frac{1}{4}$. When the unit saturates on the wrong side — y close to 0 while $t = 1$ — this factor is tiny, and the gradient vanishes exactly when the error is largest. The cross-entropy cancels the factor and gives $y - t$, which is large whenever the prediction is badly wrong. The cross-entropy is also the correct likelihood for a Bernoulli target, while the squared loss corresponds to Gaussian noise on a probability, a model that makes no sense for 0/1 labels.

> **Worked example — 6. A saturated output unit**
>
> A sigmoid output unit has $a = -4$ for an example with $t = 1$, so $y = \sigma(-4) = 0.0180$: the network is confidently wrong. With the squared loss, (10) gives
>
> $$\frac{\partial \ell}{\partial a} = (0.0180 - 1) \times 0.0180 \times 0.9820 = -0.0173;$$
>
> with the cross-entropy, $\partial \ell / \partial a = 0.0180 - 1 = -0.982$, about 57 times larger. A gradient step on the squared loss barely moves a, and the example is corrected slowly, if at all. At $a = 0$ ($y = \frac{1}{2}$) the two gradients are -0.125 and -0.5: even away from saturation the squared loss is four times weaker.

Compute losses from the pre-activations. In code, the output activation and the loss are evaluated together from the *logits* $\mathbf{a}$, never from the probabilities. For a binary output,

$$\ell = -t \log \sigma(a) - (1-t) \log\big(1 - \sigma(a)\big) = t \,\mathrm{softplus}(-a) + (1-t)\,\mathrm{softplus}(a) = \mathrm{softplus}(a) - ta,$$

with $\mathrm{softplus}(a) = \log(1 + e^{a})$, using $-\log \sigma(a) = \log(1 + e^{-a})$, $-\log(1 - \sigma(a)) = \log(1 + e^{a})$ and $\mathrm{softplus}(-a) = \mathrm{softplus}(a) - a$. The stable form $\mathrm{softplus}(a) = \max(a, 0) + \log(1 + e^{-|a|})$ never overflows. The naive route fails already for moderate logits: for $a = 20$ and $t = 0$ the loss is $\mathrm{softplus}(20) = 20.000$, but in 32-bit floats $\sigma(20) = 1 - 2 \times 10^{-9}$ rounds to exactly 1, and $-\log(1 - y)$ becomes $-\log 0 = \infty$. Likewise the multiclass loss is $\log \sum_j e^{a_j} - a_{\text{true}}$, computed after subtracting $\max_j a_j$. This is why PyTorch's `CrossEntropyLoss` and `BCEWithLogitsLoss` take logits as input.

Predicting an error bar. A regression network can output two numbers per target, a mean $\mu(\mathbf{x})$ and a log-variance $s(\mathbf{x}) = \log v(\mathbf{x})$, where v is the predicted variance, and minimise the Gaussian negative log-likelihood $\frac{1}{2}\big[(t-\mu)^2 e^{-s} + s\big]$ (constant dropped). The first term is the squared error weighted by the predicted precision, the second stops the network from claiming infinite variance everywhere. Physicists will recognise the χ^2 of Note 2, Section 3.2, with uncertainties that the network fits and that depend on the input. Such *heteroscedastic* outputs are used, for example, when a network regresses the energy of a particle shower and must say how precise each estimate is.

6 What a network can represent

6.1 Building functions from hidden units

In one dimension, a ReLU network with one hidden layer and a linear output computes a continuous function that is linear between kinks, and every such piecewise-linear function on an interval $[x_0, x_N]$ can be written in this form. Take knots $x_0 < x_1 < \cdots < x_N$ and values $f_0, f_1, \ldots, f_N$, write $s_j = (f_{j+1} - f_j)/(x_{j+1} - x_j)$ for the slope on the j-th interval, and set

$$\hat{f}(x) = f_0 + \sum_{j=0}^{N-1} c_j \,\mathrm{ReLU}(x - x_j), \qquad c_0 = s_0, \quad c_j = s_j - s_{j-1} \;\; (j \ge 1). \tag{11}$$

On $[x_0, x_1]$ only the first unit is active and $\hat{f}$ rises from f_0 with slope s_0; at each knot x_j one more unit switches on and changes the slope by c_j, to s_j. So $\hat{f}$ is the linear interpolant of the points (x_j, f_j) on $[x_0, x_N]$: N hidden units, one per knot except the last, with input weight 1, bias $-x_j$ and output weight equal to the change of slope. The hat function of Figure 5, left, is the special case $\mathrm{ReLU}(x) - 2\,\mathrm{ReLU}(x-1) + \mathrm{ReLU}(x-2)$: slopes 1, then -1, then 0.

Algorithm 2 A one-hidden-layer ReLU network that interpolates given points

Require: knots $x_0 < x_1 < \cdots < x_N$ and values $f_0, \dots, f_N$

function ReLUInterpolant($\{(x_j, f_j)\}$)
  for $j = 0, \dots, N-1$ **do**
    $s_j \leftarrow (f_{j+1} - f_j)/(x_{j+1} - x_j)$ ▷ slope of piece j
    $w_{1,j} \leftarrow 1$; $b_{1,j} \leftarrow -x_j$ ▷ unit j switches on at x_j
    $w_{2,j} \leftarrow s_j - s_{j-1}$, with $s_{-1} = 0$ ▷ change of slope
  end for
  $b_2 \leftarrow f_0$
  return $(\mathbf{w}_1, \mathbf{b}_1, \mathbf{w}_2, b_2)$ ▷ $\hat{f}(x) = \sum_j w_{2,j}\,\mathrm{ReLU}(w_{1,j}x + b_{1,j}) + b_2$
end function

Running time — constructing the interpolating network

Construction: $O(N)$, one slope and one difference per knot; no optimisation at all. *Prediction*: $O(N)$ per query, one ReLU per hidden unit (a binary search over the knots would find the active piece in $O(\log N)$, but the network evaluates all units, which on a GPU is the faster option for moderate N). *Memory*: $3N+1$ parameters. The construction is a proof device, not a training method: it needs the values of f at the knots, and in d dimensions it does not generalise beyond the grid-based bumps discussed below.

Worked example — 7. Approximating x^2 with ReLUs

Interpolate $f(x) = x^2$ on $[0, 2]$. With the three knots $0, 1, 2$ the values are $0, 1, 4$, the slopes $s_0 = 1$ and $s_1 = 3$, hence $c_0 = 1$, $c_1 = 2$ and

$$\hat{f}(x) = \mathrm{ReLU}(x) + 2\ \mathrm{ReLU}(x-1).$$

Check: $\hat{f}(1) = 1$, $\hat{f}(2) = 2 + 2 = 4$. The error is largest at the midpoints of the pieces: $\hat{f}(0.5) = 0.5$ against 0.25, $\hat{f}(1.5) = 1.5 + 1 = 2.5$ against 2.25, an error of 0.25. With knot spacing $\Delta = 0.5$ (four units, output weights $0.5, 1, 1, 1$) the maximal error is 0.0625, and with $\Delta = 0.25$ (eight units) it is 0.0156. Halving the spacing doubles the number of units and divides the error by four. In general, linear interpolation of a twice-differentiable f with spacing Δ has error at most $\frac{1}{8}\Delta^2 \max|f''|$ (here $\frac{1}{8}\Delta^2 \cdot 2 = \Delta^2/4$), so N units on a fixed interval give an error of order $1/N^2$. Figure 5, centre, shows the same for $f(x) = \sin 2x + x/2$ on $[0, \pi]$: maximal error 0.211 with 4 pieces and 0.033 with 12.

Bumps in more dimensions. In one dimension three ReLU units make a hat. In d dimensions every hidden unit is a ridge that extends without limit along $d-1$ directions, so a localised bump needs several of them. Figure 6 builds one in two dimensions from sigmoid ridges. A single unit is a smoothed cliff along its direction $\mathbf{w}$. Two units with the same direction and opposite output weights, $\sigma(4(x_1 + 1)) - \sigma(4(x_1 - 1))$, give a band, $-1 < x_1 < 1$, running parallel to the x_2-axis. Adding the analogous band $-1 < x_2 < 1$ gives a sum close to 2 in the central square, close to 1 on the four arms of a cross, and close to 0 elsewhere. The output sigmoid matters here: with threshold 1.5 it is a second non-linearity that cuts away the arms and keeps only the square. Four hidden units and an output

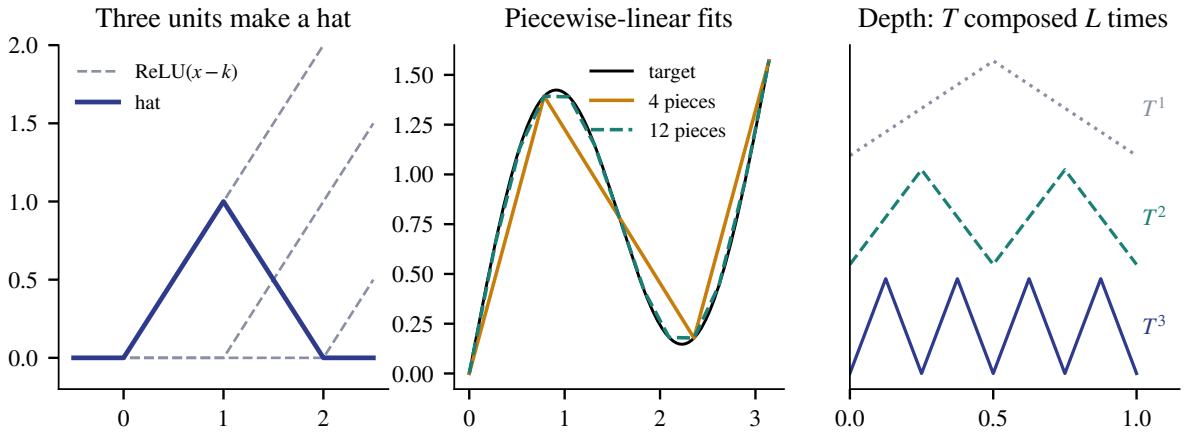


Figure 5. Left: three ReLU units (dashed) combine into a hat. Centre: one-hidden-layer ReLU networks that interpolate $\sin 2x + x/2$ at 5 and 13 equally spaced knots. Right: the tent map $T(x) = 2\,\mathrm{ReLU}(x) - 4\,\mathrm{ReLU}(x - \frac{1}{2})$ and its compositions $T^2 = T \circ T$ and T^3, offset vertically; T^L has 2^L linear pieces (Worked example 8).

unit thus make a bump, which can be placed anywhere and made as narrow as desired by scaling the weights. In d dimensions a bump needs $2d$ ridge units, two per coordinate, and approximating a general function to accuracy ϵ by bumps on a grid of spacing Δ needs about $(1/\Delta)^d$ bumps. For a smooth function and $\epsilon \sim \Delta^2$ that is $\epsilon^{-d/2}$ bumps: the curse of dimensionality of Note 3, Section 3, in a new form. Constructions of this kind prove what a network *can* represent; they say nothing about whether that is economical, or whether training finds it.

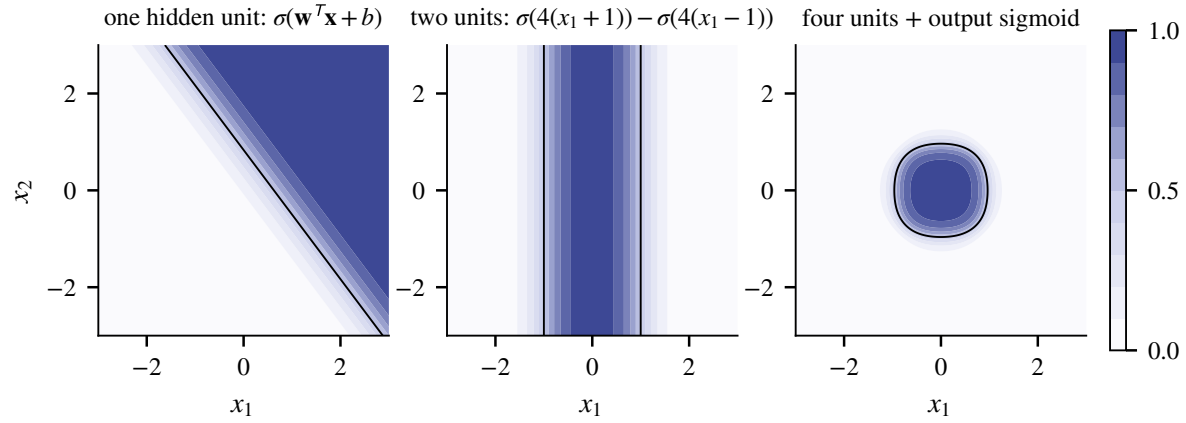


Figure 6. Building a bump from ridges. Left: one sigmoid unit, $\sigma\big(4(0.8x_1 + 0.6x_2 - 0.5)\big)$; its level sets are parallel lines. Centre: the difference of two parallel units is a band. Right: two orthogonal bands (four hidden units) followed by an output sigmoid, $\sigma\big(8(s - 1.5)\big)$ with s the sum of the two bands, give a localised bump. Black curves: level $\frac{1}{2}$.

6.2 The universal approximation theorem

The constructions above extend to a general theorem, proved for sigmoid units by Cybenko [5] and by Hornik, Stinchcombe and White [16], and later for essentially every activation [17, 20].

Universal approximation Let $g : \mathbb{R} \to \mathbb{R}$ be continuous and *not a polynomial*. For every continuous function f on a compact (closed and bounded) set $K \subset \mathbb{R}^d$ and every $\epsilon > 0$ there exist a width N and parameters $v_j, b_j \in \mathbb{R}$, $\mathbf{w}_j \in \mathbb{R}^d$ such that

$$\sup_{\mathbf{x} \in K} \left| f(\mathbf{x}) - \sum_{j=1}^{N} v_j \, g\big(\mathbf{w}_j^{\mathsf{T}} \mathbf{x} + b_j\big) \right| < \epsilon.$$

A network with one hidden layer and a linear output can approximate any continuous function on a bounded region arbitrarily well, provided it has enough hidden units.

Why "not a polynomial". If g is a polynomial of degree r, then each $g(\mathbf{w}_j^{\mathsf{T}}\mathbf{x} + b_j)$ is a polynomial of degree at most r in $\mathbf{x}$, and so is any sum of them: the network can never leave this finite-dimensional space, and cannot approximate, say, e^x uniformly on an interval when r is fixed. Any non-polynomial continuous activation — sigmoid, tanh, ReLU, GELU, even cos — escapes this trap. The theorem extends to vector outputs (apply it to each component) and to classification: decision regions of any shape can be approximated, which answers the question raised by XOR.

What the theorem does not say. It is an existence result, and its limitations matter as much as its statement. (i) The required width N can be enormous: the bump construction needs a number of units exponential in d. Barron [1] showed that for functions whose Fourier transform $\hat{f}(\omega)$ satisfies $\int \|\omega\| \, |\hat{f}(\omega)| \, d\omega < \infty$ (roughly: f is not dominated by high frequencies) the squared error of the best width-N network decreases like $1/N$, with a constant that depends on that moment but not explicitly on d — a hint that networks can beat grids on suitable function classes. (ii) It says nothing about *finding* the weights: training minimises a non-convex objective by gradient descent (Section 7.3; Note 12, Section 9). (iii) It says nothing about *generalisation*: with finite data, a network wide enough to approximate anything can also fit the noise (Note 4, Section 5). Universality is therefore not what distinguishes neural networks: polynomials, Fourier series and Gaussian-kernel machines (Note 10) are universal too. What matters in practice is how efficiently a family represents the functions that occur in real problems, and how well gradient training finds them.

6.3 Depth versus width

If one hidden layer suffices in principle, why are practical networks deep? Because composition can be exponentially more economical than addition.

Worked example — 8. The tent map: depth beats width

The tent map on $[0, 1]$ rises linearly from $T(0) = 0$ to $T(\frac{1}{2}) = 1$ and falls back to $T(1) = 0$. It is a ReLU network with two hidden units:

$$T(x) = 2\,\mathrm{ReLU}(x) - 4\,\mathrm{ReLU}\left(x - \tfrac{1}{2}\right) = \begin{cases} 2x, & 0 \le x \le \frac{1}{2}, \\ 2x - 4(x - \frac{1}{2}) = 2 - 2x, & \frac{1}{2} \le x \le 1. \end{cases}$$

Compose it with itself. $T^2 = T \circ T$ sends $[0, \frac{1}{4}]$ to $[0, 1]$, $[\frac{1}{4}, \frac{1}{2}]$ back to $[0, 1]$ (downwards), and so on: two teeth, four linear pieces. For example $T(0.3) = 0.6$, $T^2(0.3) = T(0.6) = 0.8$ and $T^3(0.3) = T(0.8) = 0.4$. Each composition doubles the number of pieces: every piece is mapped onto $[0, 1]$ and folded once more. Hence T^L has 2^L linear pieces and 2^{L-1} teeth (Figure 5, right), and it is computed by a network of L hidden layers of width 2: $2L$ hidden units in all.

A network with *one* hidden layer of N ReLU units computes $\sum_j v_j \,\mathrm{ReLU}(w_j x + b_j) + c$, whose slope can change only at the N points $x = -b_j/w_j$; it has at most $N + 1$ linear pieces. To represent T^L exactly it therefore needs $N \geq 2^L - 1$ units. For $L = 10$: 20 hidden units in ten layers, against at least 1 023 in one layer. Telgarsky [34] turned this example into a theorem: for every k there are functions computed by ReLU networks with of the order of k^3 layers of constant width that no network with at most k layers can even approximate unless it has of the order of 2^k units.

The number of linear pieces is a crude but telling measure of capacity: for networks with many inputs, the maximal number of linear regions into which a ReLU network cuts its input space grows exponentially with depth and only polynomially with width [24]. Trained networks use only a fraction of this potential, and the practical benefit of depth is better described as *reuse*: a feature computed once in a lower layer is shared by every unit above it, much as the tent map reuses the folding of the previous layer. Functions that occur in nature are often compositional — local pieces combined into larger structures — and deep networks match that structure.

In physics — networks as variational ansätze and surrogates

Universal approximation is the reason networks have become a standard *ansatz* in computational physics. Carleo and Troyer represented the ground-state wave function of a quantum spin system as a small neural network of the spin configuration and optimised its parameters variationally, reaching accuracies competitive with established methods on benchmark models [4]. Behler and Parrinello wrote the energy of a configuration of atoms as a sum of atomic energies, each computed by an MLP from rotation- and translation-invariant descriptors of the atom's neighbourhood [2]: the same physical idea as the kernel potentials of Note 10, with learned instead of fixed features. *Physics-informed neural networks* [29] go one step further and let an MLP represent the solution of a differential equation itself (Note 12, Section 10.4). In each case the theorem guarantees that a good enough network exists; whether a small one exists and can be found is an empirical question for each problem.

7 Training and capacity

7.1 The training problem

Training a network is regularised empirical risk minimisation (Note 4, Section 3.2; Note 7, Section 1):

$$J(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell\big(f_\theta(\mathbf{x}_i), \mathbf{t}_i\big) + \lambda \sum_k \left\| W^{(k)} \right\|_F^2 , \tag{12}$$

where ℓ is the per-example loss of Table 2, the penalty sums over all weight matrices, and $\|W\|_F^2 = \sum_{jk} W_{jk}^2$. The penalty is called *weight decay*; it usually leaves the biases out (Note 12, Section 6.1). Note 12 writes it as $\frac{\lambda}{2}\sum_k \left\|W^{(k)}\right\|_F^2$, so its λ is twice ours. There is no closed-form minimiser and J is not convex (Section 7.3), so it is minimised iteratively: mini-batch stochastic gradient descent, momentum or Adam (Note 7, Sections 5.3, 6.1 and 6.4), with a learning-rate schedule such as warm-up followed by cosine decay (Note 7, Section 3.3). The parameters start from small random values — not zeros, and not all equal (Note 12, Section 7) — whose scale matters a great deal. Every step needs the gradient $\nabla_\theta J$ on a mini-batch. For a network with p parameters, finite differences would cost $O(p)$ forward passes (Note 7, Section 6.5); *backpropagation* obtains the exact gradient with a backward pass that costs about twice a forward pass, independently of p, and is derived in Note 12, Section 3.

The complete loop (shuffled mini-batches, a gradient step with weight decay, early stopping with patience) is the training loop of Note 12, Section 5; its stopping rule is the one of Note 7, Section 7.1. For the digit network of Worked example 5 one epoch over $n = 60\,000$ images costs about $6np \approx 2\times10^{11}$ floating-point operations (two per weight and example forward, about twice that backward), a matter of seconds on a GPU.

7.2 Hyperparameters that control capacity

The weights are fitted on the training set; the choices around them are hyperparameters (Note 4, Section 7). For an MLP the main ones that set the *capacity* are

- the *width* H_k of the hidden layers and the *depth* L: more units, more functions;
- the *weight-decay* strength λ: large weights are needed for sharp transitions (the slope of $\sigma(ca)$ at the origin is $c/4$), so penalising them favours smooth functions;
- the *number of training epochs*: started from small weights, a tanh or sigmoid network is nearly linear (tanh $a \approx a$ for small a), and a ReLU network computes a small, gently varying function; the function becomes more complex as the weights grow during training, so stopping early limits the capacity actually used (Note 7, Section 7.1);

and, less directly, the activation function and the optimisation settings (learning rate, mini-batch size, schedule), which change which of the many minimisers training reaches. They are chosen on a validation set exactly as in Note 4, Section 7.1 (with k-fold cross-validation when data are scarce, Note 4, Section 4.2), using the criterion that matters, usually the error rate rather than the training cross-entropy. Grid and random search are Note 4, Section 7.2; advice specific to networks is in Note 12, Section 8.

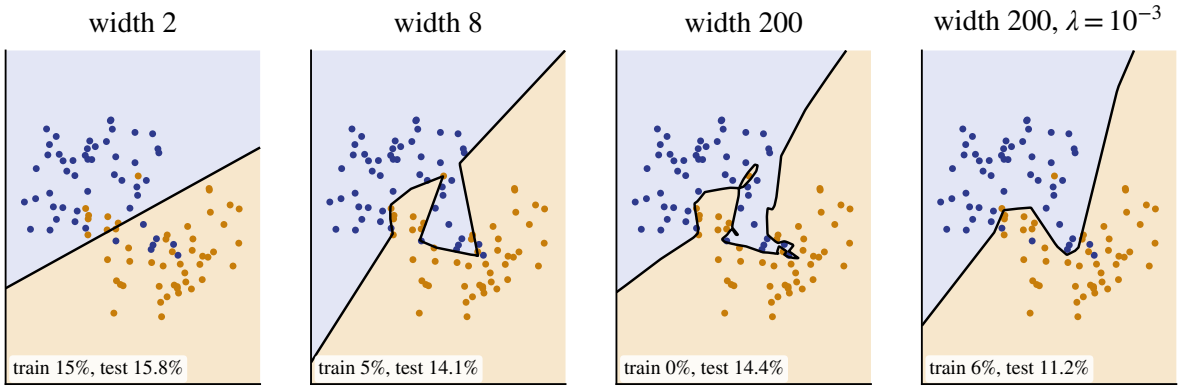


Figure 7. One-hidden-layer ReLU networks on 100 noisy two-moons points, trained by 4000 full-batch Adam steps (learning rate 0.01) on the mean cross-entropy, plus $\lambda \sum_k \|W^{(k)}\|_F^2$ in the last panel. Test errors on 5000 new points.

Figure 7 varies two of these knobs on the two-moons data of Note 10. With 2 hidden units the network can only bend its boundary once (15% training, 15.8% test error): it underfits. With 8 units the boundary follows the moons (5%, 14.1%). With 200 units and no penalty, training reaches 0% error by drawing a contorted boundary around individual points, and the test error rises to 14.4%: overfitting, the U-shaped curve of Note 4, Section 5. The same 200 units with weight decay $\lambda = 10^{-3}$ give a smooth boundary, 6% training and the best test error, 11.2%. A large network with suitable regularisation beat every unregularised width here, which reflects modern practice: choose a generous architecture and control its capacity by regularisation and early stopping, rather than by making the network small.

Figure 8 shows the number of epochs at work. The training cross-entropy decreases throughout, to 0.051 after 3000 epochs. The validation cross-entropy reaches its minimum, 0.291, at epoch 230 and then grows to 1.46, as the network becomes ever more confident on points it has memorised; early stopping on the validation cross-entropy, as in the training loop of Note 12, Section 5, keeps the parameters of epoch 230 and gives a test error of 11.5%, against 12.8% at the end. Monitoring the validation error instead would keep the parameters of epoch 340 (validation error 11.5%, test error 10.9%). The validation *error* is flatter and noisier than the validation cross-entropy: the cross-entropy punishes confident mistakes heavily, whereas the error only counts them. Which of the two to monitor is itself a choice; the error is what we finally care about, the cross-entropy is less noisy.

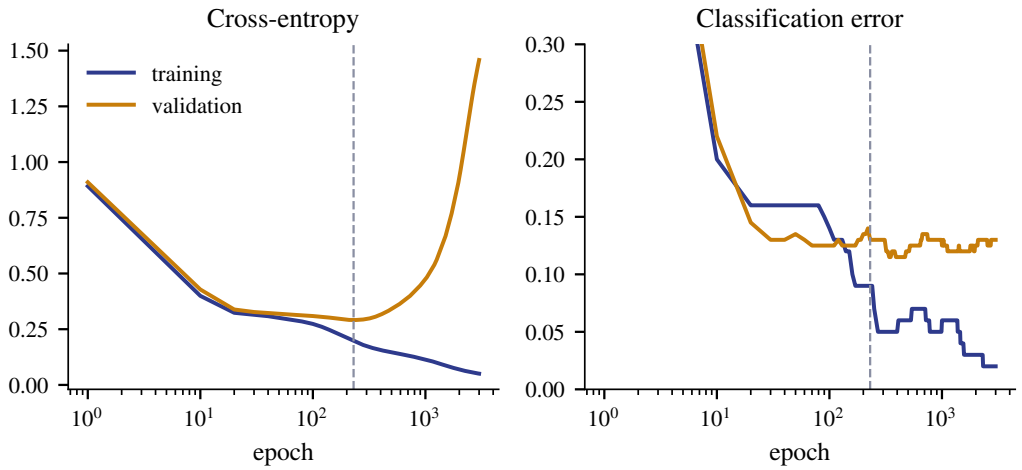


Figure 8. Training of the width-200 network of Figure 7 without weight decay (full-batch Adam, learning rate 0.003), monitored on 200 validation points. The dashed line marks the epoch of lowest validation cross-entropy, where early stopping would keep the parameters.

Modern connection — over-parameterised networks

The width-200 network has 801 parameters for 100 training points and can fit them perfectly; networks used in practice are often far more over-parameterised. Zhang and co-authors showed that standard image networks can reach zero training error even on randomly relabelled data, so their capacity in the classical sense is enormous, yet on real labels the same networks generalise well [37]. The double-descent curve of Note 4, Section 8 is one facet of this behaviour. In practice large models are trained with AdamW (Note 7, Section 6.4), whose decoupled weight decay is one of the explicit regularisers, together with early stopping, data augmentation (Note 13, Section 6) and dropout (Note 12, Section 6.3); the implicit preferences of the optimiser itself contribute as well, and how much each matters is an active research question.

7.3 A non-convex objective

Logistic regression has a convex loss, so every local minimum is a global one (Note 9, Section 4.3). A network with a hidden layer does not. Permuting the hidden units of a layer — swapping rows j and k of W_1 and of $\mathbf{b}_1$ together with columns j and k of W_2 — changes the parameter vector but not the function, so every minimiser comes with $H_1!$ copies (more for tanh units, which can flip sign, and a continuous family for ReLU units, whose weights in and out can be scaled by $c > 0$ and $1/c$). The average of two such copies gives units j and k identical incoming weights, so they compute the same feature and the network has effectively lost a unit; in general this midpoint is worse, which a convex J would not allow. Note 12, Section 9 treats the consequences for training.

In physics — degenerate minima and broken symmetry

The loss of a network is invariant under the permutation group S_{H_1} of each hidden layer (and, for tanh units, under the sign flips $\mathbb{Z}_2^{H_1}$), just as the energy of an isotropic ferromagnet is invariant under rotations of all spins. In both cases the minima are degenerate: a whole orbit of equivalent ground states, of which the system selects one. Training breaks the symmetry through its random initial condition, as a small stray field selects the magnetisation direction of a cooling magnet. Why this symmetry must be broken by a random initialisation, and not left to the training, is shown in Note 12, Section 7.

8 Neural networks are back

Neural networks have fallen out of fashion and returned more than once. A short chronology places the ideas of this note:

1943	McCulloch and Pitts describe neurons as threshold logic units [22].
1958	Rosenblatt's perceptron learns its weights from examples [31] (Note 9, Section 3).
1969	Minsky and Papert analyse what single-layer perceptrons cannot compute, XOR among it [23] (Note 9, Section 6); interest declines.
1986	Rumelhart, Hinton and Williams popularise backpropagation for training multilayer networks [32] (reverse-mode differentiation itself is older).
1989	Universal approximation theorems [5, 16]; LeCun and co-workers train a convolutional network to read handwritten zip codes [19].
1995–2005	Support vector machines and kernel methods (Note 10), with convex training and few hyperparameters, dominate much of machine learning.
2006	Layer-wise pre-training makes deep networks trainable again [13].
2010–11	Rectified linear units [26, 8].
2012	AlexNet, a deep convolutional network trained on GPUs, wins the ImageNet challenge by a wide margin [18] (Note 13, Section 7).
2015–16	Residual networks with over a hundred layers [11] (Note 12, Section 10.3).
2017	The transformer [36], whose layers alternate attention with the MLP blocks of Section 4.2.

The methods of 2012 differ from those of 1989 less than one might think: an MLP or convolutional network, a cross-entropy loss, gradients by backpropagation, stochastic gradient descent. What changed was the scale of data (the ImageNet training set has about 1.2 million labelled images), the arrival of GPUs, which made the matrix products of Algorithm 1 tens of times faster, and a set of practical improvements: non-saturating activations, careful initialisation, normalisation layers, residual connections, dropout, and software that computes gradients automatically (Note 12, Section 3.5). Most of these are the subject of Note 12. The appeal of the kernel methods of the 1990s was a convex problem with a unique answer; the success of networks after 2012 showed that a non-convex problem whose many minima are all good enough can be the better bargain.

Practical recipes, from input standardisation and the scale of the initial weights to a learning-rate search and a debugging checklist, are collected in Note 12, Section 10.

Summary An artificial neuron computes $g(\mathbf{w}^\mathsf{T}\mathbf{x} + b)$; with a sigmoid it is logistic regression and draws a hyperplane. Without non-linear activations stacked layers collapse to one linear map. Sigmoid and tanh saturate; ReLU, GELU and SiLU do not for positive inputs and dominate modern practice; the softmax turns scores into probabilities. A multilayer perceptron computes $\mathbf{h}^{(k)} = g(W^{(k)}\mathbf{h}^{(k-1)}+\mathbf{b}^{(k)})$ layer by layer, a linear model on learned features; it has $\sum_k H_k(H_{k-1}+1)$ parameters, and its forward pass costs $O(p)$ per example. Two hidden units solve XOR. The output unit and the loss are chosen together by maximum likelihood (identity and squared error, sigmoid and binary cross-entropy, softmax and cross-entropy), and then the gradient with respect to the output pre-activation is $\mathbf{y} - \mathbf{t}$. One hidden layer is a universal approximator, but may need exponentially many units; depth can represent some functions exponentially more compactly. The training objective is regularised empirical risk, non-convex because of the symmetries of hidden units; width, depth, weight decay and the number of epochs set the capacity and are chosen on validation data. Gradients come from backpropagation (Note 12).

Questions and exercises

Questions are sorted by kind. Parts A and B are of the kind used in the midterm examination: they test understanding, interpretation and short calculations (at most simple derivatives), not long proofs. Parts C and D go further and are recommended for practice and for the project.

A. Conceptual questions

A1. Which three classical models are single artificial neurons, and which activation function gives each? Why is the decision boundary of a sigmoid neuron a hyperplane even though σ is non-linear?

A2. Explain why a network in which every activation is the identity is no more expressive than a single linear layer, however many layers and units it has.

A3. What does it mean that an activation function saturates? Which functions of Table 1 saturate, and why does saturation slow down gradient-based training?

A4. What is a dead ReLU unit? How can it arise during training, and how does the leaky ReLU avoid it?

A5. What does dividing the scores of a softmax by a temperature $T > 1$ do to its output? In knowledge distillation, why can a student network learn more from the softened outputs of a teacher than from one-hot labels?

A6. In Worked example 3, the hidden layer maps $(0, 1)$ and $(1, 0)$ to the same point. Explain why this makes the problem linearly separable in the hidden space, and why no hidden layer with a *linear* activation could achieve this.

A7. For the one-hidden-layer MLP of (5), say which parts are chosen by hand and which are fitted to the data. Explain why the cost of one of its predictions, unlike that of a kernel method (Note 10), does not grow with the number n of training examples.

A8. How many parameters does a one-hidden-layer MLP with d inputs, H_1 hidden units and m outputs have? Which term dominates for image inputs, and what does this suggest about dense layers on raw pixels?

A9. Algorithm 1 processes a mini-batch with one matrix product per layer instead of a loop over the examples. Why does this not change the number of operations, and why is it nevertheless much faster on a GPU?

A10. For each of the following tasks, choose the output unit and the loss, and justify the choice by a probabilistic model of the target: (a) predicting the energy of a particle shower; (b) deciding whether an event is signal or background; (c) identifying which of five particle types produced a track; (d) tagging an image with any subset of 20 labels; (e) predicting the number of photons detected in a time window.

A11. Explain why a sigmoid output combined with the squared loss can learn very slowly from examples it gets badly wrong, while the cross-entropy does not have this problem.

A12. State the universal approximation theorem. Give three things it does *not* guarantee. Why must the activation not be a polynomial?

A13. Using the tent map, explain why a deep network can represent some functions with far fewer units than a network with one hidden layer.

A14. Name four hyperparameters that control the capacity of an MLP, and for each explain in which direction it moves the network between underfitting and overfitting.

A15. In Figure 8, the validation cross-entropy rises sharply after epoch 230 while the validation error changes little. Explain how this can happen, and discuss which of the two quantities you would use for early stopping.

A16. Why is the training objective of a network with a hidden layer not convex? Explain the role of the permutation symmetry of the hidden units.

A17. True or false, with a justification: “Because one hidden layer is a universal approximator, deep networks are never needed.”

A18. Give three reasons why neural networks returned to the front of machine learning after 2012, although their basic ingredients were known in 1989.

B. Short calculations

B1. A neuron has weights $\mathbf{w} = (1, -2, 0.5)$ and bias $b = 0.5$. For the input $\mathbf{x} = (1, 0.5, 2)$ compute the pre-activation and the outputs of a sigmoid, a tanh and a ReLU unit. Verify numerically that $\tanh a = 2\sigma(2a) - 1$ for your value of a. Give a direction $\mathbf{u} \neq \mathbf{0}$ along which the output does not change.

B2. Using step units $g(a) = \mathbf{1}\{a > 0\}$, write down the weights and biases of single neurons computing AND, OR and NOT on binary inputs. Combine them into a network with one hidden layer of two units that computes XOR, and check it on the four inputs. Then design a network computing XNOR (true when $x_1 = x_2$).

B3. An MLP has $d = 100$ inputs, two hidden layers of 64 units and 10 outputs. Compute its number of parameters, the number of multiply–adds of one forward pass, and the memory needed in 32-bit floats for the parameters alone and for parameters, gradient and Adam state together.

B4. A three-class softmax output has logits $\mathbf{a} = (1, 3, 0)$ and the true class is the third. Compute the probabilities, the cross-entropy and its gradient $\mathbf{y} - \mathbf{t}$ with respect to the logits, and recompute

the probabilities at temperature $T = 2$. Then, using the physics box of Section 2: a fermion level lies 0.05 eV above the chemical potential at $k_B T = 0.025$ eV; write its Fermi–Dirac occupation as $\sigma(a)$, and give a and the occupation.

B5. A sigmoid output unit has pre-activation $a = 3$ for an example with $t = 0$. Compute $\partial \ell / \partial a$ for the squared loss $\frac{1}{2}(y - t)^2$ and for the binary cross-entropy, and their ratio. Repeat for $a = 0$.

B6. Write the piecewise-linear interpolant of the points $(0, 0)$, $(1, 2)$, $(2, 1)$, $(3, 3)$ as a one-hidden-layer ReLU network (Algorithm 2): give all weights and biases, and check the values at the four knots and at $x = 1.5$. Then write $|x|$ (for all real x) and $\min(x, 1)$ (for $x \geq 0$) as ReLU networks with two units each.

B7. For the tent map $T(x) = 2\,\mathrm{ReLU}(x) - 4\,\mathrm{ReLU}(x - \frac{1}{2})$ compute $T(x)$ and $T^2(x)$ at $x = 0.1$, 0.3 and 0.45. How many linear pieces does T^4 have on $[0, 1]$? How many hidden units does a deep network need to compute it, and at least how many a network with one hidden layer?

B8. A network with $W_1 = \begin{pmatrix} 1 & 0 \\ 1 & -1 \\ 0 & 1 \end{pmatrix}$, $\mathbf{b}_1 = (0, 0, -1)$, ReLU hidden units, output weights $\mathbf{w}_2 = (1, -1, 2)$, output bias 0.5 and an identity output receives the mini-batch $\mathbf{x}_1 = (1, 2)$, $\mathbf{x}_2 = (2, 1)$ with targets $t_1 = 2, t_2 = 1$. Compute A_1, H, the outputs and the mean squared loss $\frac{1}{2n'} \sum_i (y_i - t_i)^2$, in the matrix form (7).

C. Derivations

C1. Prove $\tanh a = 2\sigma(2a) - 1$, and derive $\sigma' = \sigma(1 - \sigma)$ and $\tanh' = 1 - \tanh^2$. Show that every one-hidden-layer network with tanh hidden units and a linear output computes the same function as a network with sigmoid hidden units and a linear output, and give the transformation of the weights and biases.

C2. Show that a composition of L affine maps is affine. Then show that a network with identity activations and a hidden layer of width $r < \min(d, m)$ represents exactly the affine maps $\mathbf{x} \mapsto M\mathbf{x} + \mathbf{c}$ with $\mathrm{rank}\, M \leq r$.

C3. Derive $\partial \ell / \partial a = y - t$ for the sigmoid with binary cross-entropy and for the exponential output with the Poisson loss $\ell = y - t \log y$, and derive (10). Show that the binary cross-entropy can be written as $\mathrm{softplus}(a) - ta$ and that $\mathrm{softplus}(a) = \max(a, 0) + \log(1 + e^{-|a|})$.

C4. Prove that the network (11) interpolates the points (x_j, f_j), and that a one-hidden-layer ReLU network with N units on $\mathbb{R}$ has at most $N + 1$ linear pieces. Using the error bound $\frac{1}{8}\Delta^2 \max |f''|$ of linear interpolation (spacing Δ), find how many units are needed to approximate $\sin x$ on $[0, 2\pi]$ to accuracy 10^{-3}.

D. Programming

D1. Implement Algorithm 1 in numpy for arbitrary widths, ReLU hidden units and a softmax output, using only matrix operations. Check it against a loop over examples and units on random inputs, reproduce Worked example 4, and measure how the time of a forward pass grows with the width and with the mini-batch size.

D2. Using PyTorch (`torch.nn.Linear`, `torch.nn.CrossEntropyLoss` or `BCEWithLogitsLoss` and `torch.optim.AdamW`), train one-hidden-layer networks on noisy two-moons data for

widths $2, 8, 32, 200$ and several weight-decay values. Reproduce Figures 7 and 8 qualitatively, choose the width, the weight decay and the number of epochs on a validation set, and report the test error of your final choice.

D3. (a) Implement Algorithm 2 and plot the maximal error of the interpolating network for $f(x) = \sin 2x + x/2$ against the number of units on log–log axes; confirm the slope -2. (b) Train one-hidden-layer ReLU networks of the same widths by gradient descent on 200 samples of f and compare their errors with the construction. (c) Build the tent-map network T^L for $L = 1, \dots, 8$ and try to fit T^6 with a one-hidden-layer network of 64 units by training. What happens?

Further reading. Bishop [3], sections 5.1–5.2; Goodfellow, Bengio and Courville [9], chapter 6; Prince [27], chapters 3 and 4, for an excellent visual treatment of shallow and deep ReLU networks and their linear regions; Murphy [25], chapter 13.

References

[1] A. R. Barron, Universal approximation bounds for superpositions of a sigmoidal function, *IEEE Trans. Inf. Theory* 39, 930–945 (1993).

[2] J. Behler and M. Parrinello, Generalized neural-network representation of high-dimensional potential-energy surfaces, *Phys. Rev. Lett.* 98, 146401 (2007).

[3] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer (2006).

[4] G. Carleo and M. Troyer, Solving the quantum many-body problem with artificial neural networks, *Science* 355, 602–606 (2017).

[5] G. Cybenko, Approximation by superpositions of a sigmoidal function, *Math. Control Signals Systems* 2, 303–314 (1989).

[6] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, BERT: pre-training of deep bidirectional transformers for language understanding, *NAACL* (2019).

[7] S. Elfwing, E. Uchibe and K. Doya, Sigmoid-weighted linear units for neural network function approximation in reinforcement learning, *Neural Netw.* 107, 3–11 (2018).

[8] X. Glorot, A. Bordes and Y. Bengio, Deep sparse rectifier neural networks, *AISTATS* (2011).

[9] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press (2016).

[10] K. He, X. Zhang, S. Ren and J. Sun, Delving deep into rectifiers: surpassing human-level performance on ImageNet classification, *ICCV* (2015).

[11] K. He, X. Zhang, S. Ren and J. Sun, Deep residual learning for image recognition, *CVPR* (2016).

[12] D. Hendrycks and K. Gimpel, Gaussian error linear units (GELUs), arXiv:1606.08415 (2016).

[13] G. E. Hinton, S. Osindero and Y.-W. Teh, A fast learning algorithm for deep belief nets, *Neural Comput.* 18, 1527–1554 (2006).

[14] G. Hinton, O. Vinyals and J. Dean, Distilling the knowledge in a neural network, arXiv:1503.02531 (2015).

[15] J. J. Hopfield, Neural networks and physical systems with emergent collective computational abilities, *Proc. Natl. Acad. Sci. USA* 79, 2554–2558 (1982).

[16] K. Hornik, M. Stinchcombe and H. White, Multilayer feedforward networks are universal approximators, *Neural Netw.* 2, 359–366 (1989).

[17] K. Hornik, Approximation capabilities of multilayer feedforward networks, *Neural Netw.* 4, 251–257 (1991).

[18] A. Krizhevsky, I. Sutskever and G. E. Hinton, ImageNet classification with deep convolutional neural networks, *NeurIPS* (2012).

[19] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard and L. D. Jackel, Backpropagation applied to handwritten zip code recognition, *Neural Comput.* 1, 541–551 (1989).

[20] M. Leshno, V. Ya. Lin, A. Pinkus and S. Schocken, Multilayer feedforward networks with a nonpolynomial activation function can approximate any function, *Neural Netw.* 6, 861–867 (1993).

[21] A. L. Maas, A. Y. Hannun and A. Y. Ng, Rectifier nonlinearities improve neural network acoustic models, *ICML Workshop on Deep Learning for Audio, Speech and Language Processing* (2013).

[22] W. S. McCulloch and W. Pitts, A logical calculus of the ideas immanent in nervous activity, *Bull. Math. Biophys.* 5, 115–133 (1943).

[23] M. Minsky and S. Papert, *Perceptrons*, MIT Press (1969).

[24] G. Montúfar, R. Pascanu, K. Cho and Y. Bengio, On the number of linear regions of deep neural networks, *NeurIPS* (2014).

[25] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press (2022).

[26] V. Nair and G. E. Hinton, Rectified linear units improve restricted Boltzmann machines, *ICML* (2010).

[27] S. J. D. Prince, *Understanding Deep Learning*, MIT Press (2023).

[28] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei and I. Sutskever, Language models are unsupervised multitask learners, OpenAI technical report (2019).

[29] M. Raissi, P. Perdikaris and G. E. Karniadakis, Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* 378, 686–707 (2019).

[30] P. Ramachandran, B. Zoph and Q. V. Le, Searching for activation functions, arXiv:1710.05941 (2017).

[31] F. Rosenblatt, The perceptron: a probabilistic model for information storage and organization in the brain, *Psychol. Rev.* 65, 386–408 (1958).

[32] D. E. Rumelhart, G. E. Hinton and R. J. Williams, Learning representations by back-propagating errors, *Nature* 323, 533–536 (1986).

[33] N. Shazeer, GLU variants improve transformer, arXiv:2002.05202 (2020).

[34] M. Telgarsky, Benefits of depth in neural networks, *COLT* (2016).

[35] H. Touvron *et al.*, LLaMA: open and efficient foundation language models, arXiv:2302.13971 (2023).

[36] A. Vaswani *et al.*, Attention is all you need, *NeurIPS* (2017).

[37] C. Zhang, S. Bengio, M. Hardt, B. Recht and O. Vinyals, Understanding deep learning requires rethinking generalization, *ICLR* (2017).

The selection and order of topics in these notes follow the course IFT 6390 *Fondements de l'apprentissage machine* at Université de Montréal (taught by Ioannis Mitliagkas, slides by Pascal Vincent). The text, examples, figures and exercises of this note are original. Figures are computed from synthetic data by `figures.py`, published with the sources of these notes.